RAG Pipeline - Complete Theory Notes (Step by Step)

Step 1: Define the Use Case

What is it?

Before building a RAG pipeline, you must clearly define **what problem you're solving**. This determines every technical decision downstream — data sources, chunking strategy, embedding model, LLM choice, and evaluation metrics.

Types of RAG Use Cases

Use Case	Description	Example
Question Answering	Answer specific questions from a knowledge base	"What is our refund policy?"
Document Summarization	Summarize large documents using retrieval	"Summarize this 100-page report"
Conversational AI	Chat with context from documents	Customer support chatbot
Code Assistant	Retrieve code snippets and documentation	"How to connect to S3 in Python?"
Research Assistant	Search and synthesize across papers	"What are latest findings on X?"
Legal/Compliance	Search contracts, regulations	"What does clause 5.2 say?"

Key Decisions Based on Use Case

Decision	Simple Q&A	Conversational	Research
Chunk Size	Small (200-500 tokens)	Medium (500-1000)	Large (1000+)
Top-K	3-5	5-10	10-20
LLM	Fast (GPT-4o-mini)	Balanced (GPT-4o)	Powerful (GPT-4, Claude)
Prompt Style	Simple/Strict	Conversational	Sources/Citations
Latency Tolerance	Low (<2s)	Medium (<5s)	High (<30s)

Checklist Before Starting

- ☐ What question types will users ask?
- ☐ What data sources are available?
- ☐ What accuracy level is needed?
- ☐ What is the acceptable latency?
- ☐ Do you need source citations?
- ☐ Is it single-turn or multi-turn conversation?
- ☐ What is the scale (users, documents)?

Step 2: Collect Data Sources

What is it?

Data collection is the process of **gathering raw data** from various sources (PDFs, websites, APIs, databases, etc.) that will form the knowledge base for your RAG system.

Types of Data Sources

Source Type	Tools/Libraries	Best For
PDF	PyPDF2, pdfplumber, PyMuPDF	Reports, papers, manuals
Web Pages	BeautifulSoup, Selenium	Articles, documentation
APIs	requests, httpx	Real-time data, databases
Text Files	Built-in Python	Logs, notes, configs
DOCX	python-docx	Business documents
CSV/JSON	pandas, json	Structured data
Databases	SQLAlchemy, pymongo	Enterprise data

PDF Extraction Tools Comparison

Tool	Advantages	Disadvantages	Best For
PyPDF2	Lightweight, fast, no dependencies	Poor with complex layouts, no table extraction	Simple text PDFs
pdfplumber	Excellent table extraction, layout-aware	Slower than PyPDF2, higher memory	PDFs with tables
PyMuPDF (fitz)	Fastest, image extraction, metadata	Larger install size	Large PDFs, images
spaCy + PyPDF2	NLP processing (entities, tokens)	Needs model download, slower	When you need NLP on extracted text

Web Scraping Tools Comparison

Tool	Advantages	Disadvantages	Best For
BeautifulSoup	Simple, lightweight, fast	Can't handle JavaScript-rendered pages	Static HTML pages
Selenium	Handles dynamic/JS pages, browser automation	Slow, needs browser driver, heavy	JavaScript-heavy sites
Scrapy	Async, fast, built-in pipeline	Steeper learning curve	Large-scale scraping

Data Quality Considerations

- **Freshness**: How recent is the data?
- **Completeness**: Are there missing sections?
- **Accuracy**: Is the source reliable?
- **Format consistency**: Mixed formats need different parsers

- **Size**: Large datasets need batch processing

Industry Standards

- Always respect `robots.txt` when scraping
- Add `User-Agent` headers for web requests
- Handle rate limiting for APIs
- Store raw data before processing (never lose originals)
- Log all data sources for traceability

Step 3: Data Cleaning & Preprocessing

What is it?

Data cleaning removes noise, inconsistencies, and irrelevant content from raw text to improve the quality of embeddings and retrieval accuracy.

Types of Preprocessing Operations

Operation	What it Does	When to Use	When NOT to Use
Whitespace Removal	Removes extra spaces, tabs, newlines	Always	Never skip this
Lowercasing	Converts all text to lowercase	Keyword matching, TF-IDF	Semantic embeddings (they handle case)
URL Removal	Strips http/www links	General text cleaning	If URLs are important context
Email Removal	Strips email addresses	Privacy, PII removal	Contact information systems
Special Char Removal	Removes symbols like @#$%	Clean text for embeddings	Code documentation, math content
Number Removal	Removes standalone numbers	General text	Financial, scientific documents
Stopword Removal	Removes "the", "is", "at" etc.	TF-IDF, keyword search	Semantic embeddings (context matters)
Stemming	Reduces words to root (running→run)	Keyword search	Semantic embeddings
Lemmatization	Reduces to dictionary form (better→good)	NLP pipelines	When exact words matter

Preprocessing for Different Embedding Types

Embedding Type	Recommended Preprocessing
TF-IDF	Lowercase + stopword removal + stemming + special char removal
Sentence Transformers	Minimal — just whitespace cleanup, keep case and stopwords
OpenAI Embeddings	Minimal — whitespace cleanup only
BM25	Lowercase + stopword removal + stemming

⚠️ Common Mistakes

- **Over-cleaning**: Removing too much context hurts semantic embeddings
- **Under-cleaning**: Noise in text creates poor embeddings
- **Wrong order**: Always clean BEFORE chunking (not after)
- **Losing structure**: Removing all newlines destroys paragraph boundaries needed for chunking

Pipeline Order

```
Raw Text → Remove URLs/Emails → Clean Whitespace → (Optional: Lowercase/Stemming) → Ready for Chunking
```

Step 4: Document Parsing

What is it?

Document parsing extracts **structured information** (text, tables, headings, metadata, images) from raw files. It goes beyond simple text extraction — it understands the document's structure.

Parsing vs Collection

Aspect	Collection (Step 2)	Parsing (Step 4)
Goal	Get the file	Extract structured content
Output	Raw bytes/text	Text + tables + metadata + headings
Example	Download PDF	Extract page-wise text, tables, images

Parsing Tools by Document Type

Document Type	Tool	Extracts
PDF	PyPDF2	Text only
PDF	pdfplumber	Text + Tables
PDF	PyMuPDF	Text + Images + Metadata
HTML	BeautifulSoup	Text + Headings + Links + Structure
DOCX	python-docx	Text + Tables + Styles
Markdown	regex / mistune	Text + Headings + Code blocks
CSV	pandas	Rows + Columns + Data types
JSON	json module	Nested key-value structures

Advanced Parsing (OCR & Vision)

Tool	Use Case	Advantages	Disadvantages
Tesseract OCR	Scanned PDFs, images	Free, widely used	Low accuracy on poor scans
AWS Textract	Documents, forms, tables	High accuracy, managed service	Costs money, needs AWS
Azure Document Intelligence	Complex documents	Great table extraction	Costs money, needs Azure

Tool	Use Case	Advantages	Disadvantages
GPT-4 Vision	Any image/document	Understands context	Expensive, slow

When to Keep Parsing Separate from Collection

- Large-scale pipelines (collect 1000s of files, parse later)
- Multiple parsing strategies for same file
- Need to extract tables, images separately
- When you want to retry parsing without re-downloading

When to Combine (Simpler)

- Small projects
- Single document type
- No need for advanced extraction

Step 5: Text Chunking / Splitting

What is it?

Chunking splits large documents into smaller pieces that can be individually embedded and retrieved. This is one of the **most critical steps** — bad chunking = bad retrieval.

Types of Chunking

Method	How it Works	Chunk Size Control	Context Preservation
Character-based	Split every N characters	Exact	❌ Poor — breaks mid-sentence
Word-based	Split every N words	Approximate	⚠️ Medium — may break mid-paragraph
Sentence-based	Split every N sentences	Variable	✅ Good — respects sentence boundaries
Paragraph-based	Split on double newlines	Variable	✅ Good — respects paragraph boundaries
Token-based	Split every N tokens	Exact (for LLM)	❌ Poor — breaks mid-sentence
Semantic	Split when topic changes	Variable	✅ ✅ Best — respects meaning
Recursive	Try separators in hierarchy	Configurable	✅ Good — tries best boundary first
Hierarchical	Parent-child chunks	Multi-level	✅ ✅ Best — preserves full context

Detailed Comparison

Method	Advantages	Disadvantages	Best For
Character	Simple, predictable size	Breaks words/sentences	Quick prototyping
Word	Respects word boundaries	May break mid-sentence	General purpose
Sentence	Natural boundaries	Variable chunk sizes	Q&A systems
Paragraph	Preserves topic coherence	Very variable sizes	Well-structured documents
Token	Matches LLM token limits	Breaks mid-sentence	Token budget management

Method	Advantages	Disadvantages	Best For
Semantic	Best retrieval quality	Slow (needs embeddings), complex	Production systems
Recursive	Adapts to content structure	More complex logic	LangChain default, production
Hierarchical	Parent context available	Complex, more storage	When context is critical

Overlap (Critical Concept)

Overlap means chunks share some text at boundaries to prevent losing context at split points.

```
Without Overlap:  [Chunk 1: "AI is a field"] [Chunk 2: "of computer science"]
With Overlap:     [Chunk 1: "AI is a field of"] [Chunk 2: "a field of computer science"]
```

Overlap %	Pros	Cons	Recommended For
0%	No redundancy, less storage	Context lost at boundaries	Simple documents
10-15%	Good balance	Slight redundancy	General purpose ✅
20-30%	Strong context preservation	More storage, duplicate results	Complex documents
>30%	Maximum context	Too much redundancy, waste	Rarely needed

Chunk Size Guidelines

Chunk Size (tokens)	Retrieval Precision	Context Richness	Best For
100-200	✅ High (specific)	❌ Low	Factoid Q&A
200-500	✅ Good	✅ Good	General purpose ✅
500-1000	⚠️ Medium	✅ High	Summarization, complex topics
1000+	❌ Low (too broad)	✅ ✅ Very High	Full document context

Industry Standards

- **LangChain default**: Recursive chunking, 1000 chars, 200 overlap
- **LlamaIndex default**: Sentence-based, 1024 tokens
- **OpenAI recommendation**: 200-500 tokens per chunk
- **Best practice**: Test multiple strategies and measure retrieval quality

---

Step 6: Generate Embeddings for Each Chunk

What is it?

Embeddings convert text into **numerical vectors** (arrays of numbers) that capture semantic meaning. Similar texts produce similar vectors, enabling similarity search.

Types of Embedding Models

Model/Provider	Dimensions	Type	Cost	Quality
TF-IDF	Variable (vocab size)	Sparse, keyword-based	Free	⭐⭐ Low
BM25	Variable	Sparse, keyword-based	Free	⭐⭐ Low-Medium
Word2Vec	100-300	Dense, word-level	Free	⭐⭐⭐ Medium
all-MiniLM-L6-v2	384	Dense, sentence-level	Free	⭐⭐⭐⭐ Good
all-mpnet-base-v2	768	Dense, sentence-level	Free	⭐⭐⭐⭐ Good
OpenAI text-embedding-3-small	1536	Dense, sentence-level	Paid ($0.02/1M tokens)	⭐⭐⭐⭐⭐ Excellent
OpenAI text-embedding-3-large	3072	Dense, sentence-level	Paid ($0.13/1M tokens)	⭐⭐⭐⭐⭐ Best
Cohere embed-v3	1024	Dense, sentence-level	Paid	⭐⭐⭐⭐⭐ Excellent
Amazon Titan Embed	1536	Dense, sentence-level	Paid (AWS)	⭐⭐⭐⭐ Good
BGE-large	1024	Dense, sentence-level	Free	⭐⭐⭐⭐⭐ Excellent

Sparse vs Dense Embeddings

Aspect	Sparse (TF-IDF, BM25)	Dense (Transformers)
How it works	Keyword frequency counting	Neural network encoding
Vector size	Large (vocabulary size: 10K-100K+)	Fixed (384-3072)
Semantic understanding	❌ No — exact keyword match only	✅ Yes — understands meaning
"King" ≈ "Monarch"	❌ No match	✅ High similarity
Speed	✅ Fast to compute	⚠️ Slower (needs GPU for large scale)
Storage	⚠️ Large but sparse	✅ Compact
Best for	Keyword search, prototyping	Semantic search, production

⚠️ Critical Rules for Embeddings

1. **Same model for chunks and queries** — You MUST use the same embedding model for indexing and querying
2. **TF-IDF cannot batch** — Must fit on ALL texts at once (single vocabulary)
3. **Dense models can batch** — Process in batches of 32-64 for efficiency
4. **Dimension must match** — All embeddings stored together must have same dimensions

Embedding Dimension Trade-offs

Dimension	Storage	Speed	Quality	Use Case
384	✅ Small	✅ Fast	⭐⭐⭐ Good	Prototyping, small datasets

Dimension	Storage	Speed	Quality	Use Case
768	⚠️ Medium	⚠️ Medium	⭐⭐⭐⭐ Better	General production
1536	❌ Large	❌ Slower	⭐⭐⭐⭐⭐ Best	High-quality production
3072	❌❌ Very Large	❌❌ Slowest	⭐⭐⭐⭐⭐ Best	When quality is top priority

Batch Processing

- **Why batch?** — Sending 1000 texts one-by-one is slow; batching is 10-50x faster
- **Typical batch size**: 32-64 for local models, 100-2000 for API models
- **Exception**: TF-IDF must process ALL texts at once (not in batches) because it builds a shared vocabulary

---

Step 7: Store Embeddings in a Vector Database

What is it?

A vector database stores embeddings and enables fast **similarity search** — finding the most similar vectors to a given query vector.

Types of Vector Databases

Database	Type	Persistence	Scalability	Cost	Best For
FAISS	Library (local)	File-based	Single machine	Free	Prototyping, small-medium datasets
ChromaDB	Embedded DB	Disk-based	Single machine	Free	Local development, easy setup
Pinecone	Cloud managed	Cloud	Unlimited	Paid	Production, enterprise scale
Weaviate	Self-hosted/Cloud	Disk/Cloud	Horizontal	Free/Paid	Hybrid search, GraphQL API
Qdrant	Self-hosted/Cloud	Disk/Cloud	Horizontal	Free/Paid	High performance, filtering
Milvus	Self-hosted	Disk	Horizontal	Free	Large-scale, billion+ vectors
pgvector	PostgreSQL extension	Disk	Single/Cluster	Free	Already using PostgreSQL

Detailed Comparison

Feature	FAISS	ChromaDB	Pinecone	Weaviate
Setup	`pip install faiss-cpu`	`pip install chromadb`	API key needed	Docker or cloud
Max vectors	~100M (RAM limited)	~1M	Unlimited	Unlimited
Metadata filtering	❌ Manual	✅ Built-in	✅ Built-in	✅ Built-in
Hybrid search	❌ No	❌ No	❌ No	✅ Yes (vector + keyword)
Auto-scaling	❌ No	❌ No	✅ Yes	✅ Yes (cloud)
Latency	⚡ <1ms	⚡ <5ms	⚡ <50ms	⚡ <50ms
Production ready	⚠️ Needs wrapper	⚠️ Growing	✅ Yes	✅ Yes

Index Types in FAISS

Index	How it Works	Speed	Accuracy	Memory	Best For
IndexFlatL2	Brute force (exact)	❌ Slow	✅ 100%	✅ Low	<100K vectors
IndexFlatIP	Brute force (inner product)	❌ Slow	✅ 100%	✅ Low	Cosine similarity
IndexIVFFlat	Clustering + search	✅ Fast	⚠️ ~95%	✅ Low	100K-1M vectors
IndexIVFPQ	Clustering + compression	✅ ✅ Fastest	⚠️ ~90%	✅ ✅ Lowest	1M+ vectors
IndexHNSW	Graph-based	✅ Fast	✅ ~99%	❌ High	Best accuracy-speed trade-off

Distance Metrics

Metric	Formula	Range	Better Score	Use Case
L2 (Euclidean)	$\sqrt{\Sigma(a-b)^2}$	0 to ∞	**Lower = better**	FAISS default, general purpose
Cosine Similarity	$(a \cdot b)/(\lVert a \rVert \cdot \lVert b \rVert)$	-1 to 1	**Higher = better**	Most common in NLP
Dot Product	$\Sigma(a \times b)$	-∞ to ∞	**Higher = better**	Normalized vectors

Score Interpretation Guide

For L2 (Euclidean) Distance — Lower is better:

Score Range	Meaning
0.0 - 0.5	✅ Excellent match (nearly identical)
0.5 - 1.0	✅ Good match (very similar)
1.0 - 1.5	⚠️ Moderate match (somewhat related)
1.5 - 2.0	⚠️ Weak match (loosely related)
> 2.0	❌ Poor match (likely irrelevant)

For Cosine Similarity — Higher is better:

Score Range	Meaning
0.9 - 1.0	✅ Excellent match (nearly identical)
0.7 - 0.9	✅ Good match (very similar)
0.5 - 0.7	⚠️ Moderate match (somewhat related)
0.3 - 0.5	⚠️ Weak match (loosely related)
< 0.3	❌ Poor match (likely irrelevant)

Industry Standards

- **Prototyping**: FAISS or ChromaDB (free, local, fast)
- **Production (small)**: ChromaDB or Qdrant
- **Production (large)**: Pinecone, Weaviate, or Milvus
- **Already on AWS**: Amazon OpenSearch with vector search, or pgvector on RDS

Step 8: Create Query Embedding for User Question

What is it?

Convert the user's question into the **same vector space** as the stored chunks, so we can compare them using similarity search.

Critical Rule

> ***The query MUST be embedded using the SAME model/provider that was used to embed the chunks.***

Chunk Embedding	Query Embedding	Works?
OpenAI text-embedding-3-small	OpenAI text-embedding-3-small	✅ Yes
HuggingFace MiniLM	HuggingFace MiniLM	✅ Yes
OpenAI	HuggingFace	❌ No — different vector spaces
TF-IDF (fit on chunks)	TF-IDF (transform with same vectorizer)	✅ Yes
TF-IDF (fit on chunks)	TF-IDF (new fit on query)	❌ No — different vocabulary

Query Enhancement Techniques

Technique	How it Works	Advantage	Disadvantage
Raw Query	Embed user question as-is	Simple, fast	Short queries have weak embeddings
Query Expansion	Add synonyms/related terms	Better recall	May add noise
HyDE	LLM generates hypothetical answer, embed that	Much better retrieval	Needs LLM call (slower, costs)
Multi-Query	Generate multiple query variations, search all	Covers more angles	Multiple searches (slower)
Step-back Prompting	Make query more general first	Better for specific questions	May lose specificity

HyDE (Hypothetical Document Embeddings) — Deep Dive

```
User Query: "What is RAG?"
     ↓
LLM generates hypothetical answer:
"RAG stands for Retrieval Augmented Generation. It is a technique that combines
information retrieval with text generation to provide more accurate responses..."
     ↓
Embed this hypothetical answer (not the original query)
     ↓
Search vector DB with this embedding
```

Why it works: The hypothetical answer is closer in vector space to actual document chunks than a short question.

Query Embedding Latency

Provider	Latency (single query)	Notes
TF-IDF	<1ms	Just a transform, no model
HuggingFace (local)	5-20ms	Depends on GPU/CPU
OpenAI API	50-200ms	Network latency
AWS Bedrock	100-300ms	Network latency

Step 9: Similarity Search in Vector Database

What is it?

Similarity search finds the **closest vectors** in the database to the query vector. This is the core retrieval mechanism in RAG.

Types of Search

Search Type	How it Works	Speed	Accuracy	Best For
Exact (Brute Force)	Compare query with ALL vectors	❌ Slow O(n)	✅ 100%	Small datasets (<100K)
ANN (Approximate)	Use index structures to skip comparisons	✅ Fast O(log n)	⚠️ 95-99%	Large datasets (100K+)
Hybrid	Vector search + keyword search combined	⚠️ Medium	✅ High	When keywords matter too

ANN (Approximate Nearest Neighbor) Algorithms

Algorithm	How it Works	Speed	Accuracy	Memory	Best For
HNSW	Builds navigable graph layers	✅ Fast	✅ ~99%	❌ High	Best overall quality
IVF	Clusters vectors, searches nearest clusters	✅ Fast	⚠️ ~95%	✅ Low	Large datasets
PQ	Compresses vectors into codes	✅✅ Fastest	⚠️ ~90%	✅✅ Lowest	Billion-scale, memory limited
LSH	Hash-based bucketing	✅ Fast	⚠️ ~85%	✅ Low	Very high dimensional
SCANN	Google's optimized ANN	✅✅ Fast	✅ ~98%	⚠️ Medium	Google-scale datasets

Similarity Metrics Deep Dive

Cosine Similarity

```
cos(A, B) = (A · B) / (||A|| × ||B||)
```

- Measures **angle** between vectors (ignores magnitude)
- Range: -1 to 1 (1 = identical direction)
- **Best for**: Text similarity (most common in NLP)
- **Why**: Document length doesn't affect similarity

Euclidean Distance (L2)

```
d(A, B) = √(Σ(Ai - Bi)²)
```

- Measures **straight-line distance** between vectors
- Range: 0 to ∞ (0 = identical)
- **Best for**: When magnitude matters
- **FAISS default**

Dot Product (Inner Product)

```
dot(A, B) = Σ(Ai × Bi)
```

- Measures **alignment** of vectors
- Range: -∞ to ∞ (higher = more similar)
- **Best for**: Normalized vectors (equivalent to cosine then)

Which Metric to Use?

Scenario	Recommended Metric	Why
General text search	Cosine Similarity	Length-invariant, standard for NLP
OpenAI embeddings	Cosine Similarity	OpenAI recommends this
FAISS default	L2 (Euclidean)	Built-in, fast
Normalized vectors	Dot Product	Equivalent to cosine, faster
When magnitude matters	Euclidean	Captures both direction and magnitude

Search Parameters

Parameter	What it Controls	Low Value	High Value
top_k	Number of results returned	Precise, fast	More context, slower
nprobe (IVF)	Number of clusters to search	Faster, less accurate	Slower, more accurate
ef_search (HNSW)	Search depth in graph	Faster, less accurate	Slower, more accurate
score_threshold	Minimum similarity to return	More results	Only high-quality results

---

Step 10: Retrieve Top-K Relevant Chunks

What is it?

After similarity search, select the **K most relevant chunks** to include as context for the LLM. The value of K directly impacts answer quality.

Top-K Selection Guidelines

K Value	Pros	Cons	Best For
1-3	Precise, low noise, fast	May miss relevant info	Factoid Q&A ("What is X?")
3-5	Good balance ✅	Slight noise possible	General purpose
5-10	Rich context	More noise, higher token cost	Complex questions
10-20	Maximum coverage	High noise, expensive, may confuse LLM	Research, summarization
20+	Exhaustive	Diminishing returns, token limit issues	Rarely needed

Industry Standard: K = 3 to 5 for most use cases

Re-Ranking After Retrieval

Initial retrieval (top-K) may not be perfectly ordered. Re-ranking improves the order.

Re-Ranking Method	How it Works	Quality	Speed	Cost
No re-ranking	Use vector DB order as-is	⭐⭐⭐	✅ Fast	Free
Keyword overlap	Count query words in chunk	⭐⭐⭐	✅ Fast	Free
Cross-Encoder	Joint model scores query+chunk pair	⭐⭐⭐⭐⭐	❌ Slow	Free (local)
Cohere Rerank	API-based reranking	⭐⭐⭐⭐⭐	⚠️ Medium	Paid
ColBERT	Token-level interaction	⭐⭐⭐⭐⭐	⚠️ Medium	Free (local)

Retrieval Pipeline Pattern

```
Query → Vector Search (top 20) → Re-rank → Take top 5 → Send to LLM
```

Why over-retrieve then re-rank?

- Vector search is fast but approximate
- Re-ranker is slow but more accurate
- Get best of both: speed + quality

Filtering Strategies

Strategy	What it Does	Example
Score threshold	Remove chunks below minimum score	Only keep cosine > 0.7
Metadata filter	Filter by source, date, type	Only from "policy.pdf"
Deduplication	Remove near-duplicate chunks	95% word overlap = duplicate
Diversity	Ensure chunks from different sections	MMR (Maximal Marginal Relevance)

MMR (Maximal Marginal Relevance)

Balances **relevance** and **diversity** — avoids returning 5 chunks that all say the same thing.

```
MMR = λ × Similarity(query, chunk) - (1-λ) × max(Similarity(chunk, already_selected))
```

- λ = 1.0 → Pure relevance (may have duplicates)

- λ = 0.5 → Balanced (recommended) ✅
- λ = 0.0 → Pure diversity (may miss relevant chunks)

Step 11: Build Prompt with Retrieved Context

What is it?

Construct a prompt that combines the user's question with the retrieved chunks, instructing the LLM how to use the context to generate an answer.

Types of Prompt Styles

Style	Structure	Best For	Risk
Simple	Context + Question	Quick answers	LLM may ignore context
Strict	"ONLY use context, say I don't know otherwise"	Factual accuracy	May refuse valid answers
Sources	Numbered sources + "Cite [Source N]"	Traceability	Longer prompts
Conversational	Chat history + Context + Question	Multi-turn chat	Context window fills fast
Custom	System prompt + Context + Question	Domain-specific	Needs prompt engineering

Prompt Template Comparison

Template	Hallucination Risk	Answer Quality	Token Usage
Simple	⚠️ Medium	⭐⭐⭐ Good	✅ Low
Strict	✅ Low	⭐⭐⭐⭐ High (when context has answer)	✅ Low
Sources	✅ Low	⭐⭐⭐⭐⭐ Best (verifiable)	⚠️ Medium
Conversational	⚠️ Medium	⭐⭐⭐⭐ Good	❌ High
Custom	Depends on prompt	Depends on prompt	Varies

Context Window Management

LLM	Context Window	Usable for Context	Reserve for Answer
GPT-4o-mini	128K tokens	~100K tokens	~2K tokens
GPT-4o	128K tokens	~100K tokens	~4K tokens
Claude 3.5 Sonnet	200K tokens	~180K tokens	~4K tokens
Llama 3 (8B)	8K tokens	~6K tokens	~2K tokens
Mistral 7B	32K tokens	~28K tokens	~2K tokens

Token Budget Formula

```
Total Tokens = System Prompt + Context Chunks + Question + Answer
                 (~100)     + (K × chunk_size) + (~50)  + (~500)
```

Example: K=5, chunk_size=200 tokens

```
100 + (5 × 200) + 50 + 500 = 1650 tokens
```

Prompt Engineering Best Practices

- **Be explicit**: "Answer ONLY from the context" reduces hallucination
- **Order matters**: Most relevant chunk first (LLMs pay more attention to start)
- **Separate context clearly**: Use markers like "Context:", "---", or XML tags
- **Limit context**: Don't stuff too many chunks — quality > quantity
- **Include format instructions**: "Answer in 2-3 sentences" or "Use bullet points"

Anti-Patterns (What NOT to Do)

- ❌ Dumping entire documents as context
- ❌ No instruction on how to use context
- ❌ Mixing irrelevant chunks with relevant ones
- ❌ Not reserving tokens for the answer
- ❌ Ignoring chat history in conversational settings

Step 12-13: Send Prompt to LLM & Generate Final Answer

What is it?

Send the constructed prompt (context + question) to a Large Language Model to generate a natural language answer grounded in the retrieved context.

Types of LLM Providers

Provider	Models	Type	Cost	Latency	Best For
OpenAI	GPT-4o, GPT-4o-mini	Cloud API	Paid	1-5s	Best quality, easy setup
AWS Bedrock	Claude, Titan, Llama	Cloud API	Paid (per token)	1-5s	AWS ecosystem, enterprise
Cohere	Command R+	Cloud API	Paid	1-3s	RAG-optimized models
HuggingFace	Flan-T5, Mistral, Llama	Local/API	Free (local)	2-10s	Privacy, no API costs
Ollama	Llama 3, Mistral, Phi	Local	Free	2-15s	Fully offline, privacy
Google	Gemini Pro, Gemini Flash	Cloud API	Paid	1-3s	Multimodal, long context

Model Selection Guide

Need	Recommended Model	Why
Cheapest + Good	GPT-4o-mini	$0.15/1M input tokens, great quality
Best quality	GPT-4o or Claude 3.5 Sonnet	Top reasoning, best accuracy
Fastest	Gemini Flash or GPT-4o-mini	Optimized for speed
Free + Local	Llama 3 8B via Ollama	No API costs, runs on laptop

Need	Recommended Model	Why
AWS native	Claude via Bedrock	Pay-per-use, no key management
Long context	Claude 3.5 (200K) or Gemini (1M)	Huge context windows
Privacy critical	Ollama + Llama/Mistral	Data never leaves your machine

LLM Parameters

Parameter	What it Controls	Low Value	High Value	RAG Recommended
temperature	Randomness/creativity	Deterministic, factual	Creative, varied	**0.0 - 0.3** (factual)
max_tokens	Maximum response length	Short answers	Long answers	256-1024
top_p	Nucleus sampling	More focused	More diverse	0.9-1.0
frequency_penalty	Reduce repetition	Allow repetition	Avoid repetition	0.0-0.5
presence_penalty	Encourage new topics	Stay on topic	Explore topics	0.0

Temperature Guide for RAG

Temperature	Behavior	Use Case
0.0	Deterministic, same answer every time	Factual Q&A, compliance ✅
0.1-0.3	Mostly deterministic, slight variation	General RAG ✅
0.5-0.7	Balanced creativity	Summarization, writing
0.8-1.0	Creative, varied	NOT recommended for RAG ❌

Cost Comparison (per 1M tokens, approximate)

Model	Input Cost	Output Cost	Quality
GPT-4o-mini	$0.15	$0.60	⭐⭐⭐⭐
GPT-4o	$2.50	$10.00	⭐⭐⭐⭐⭐
Claude 3.5 Sonnet	$3.00	$15.00	⭐⭐⭐⭐⭐
Claude 3 Haiku	$0.25	$1.25	⭐⭐⭐⭐
Gemini Flash	$0.075	$0.30	⭐⭐⭐⭐
Llama 3 (Ollama)	Free	Free	⭐⭐⭐

Streaming vs Non-Streaming

Mode	How it Works	UX	Implementation
Non-streaming	Wait for full response	User waits 2-10s, then sees full answer	Simple
Streaming	Tokens arrive one-by-one	User sees answer being typed (feels faster)	Needs SSE/WebSocket

Recommendation: Use streaming for UI applications, non-streaming for APIs.

Step 14: Post-Processing / Formatting Response

What is it?

Clean, format, and enhance the raw LLM output before presenting it to the user. This includes removing filler phrases, adding sources, and structuring the response.

Types of Post-Processing

Operation	What it Does	When to Use
Clean whitespace	Remove extra spaces/newlines	Always
Remove filler phrases	Strip "As an AI...", "Based on context..."	Always
Add source citations	Append source references	When traceability needed
Truncate	Limit response length	Chat interfaces, APIs
Format as Markdown	Add headers, bullets, code blocks	Web/UI display
Format as JSON	Structured output with metadata	API responses
Confidence scoring	Estimate answer reliability	Production systems

Output Format Comparison

Format	Best For	Contains	Example Use
Plain Text	Simple chat	Answer + sources	CLI tools, simple bots
Markdown	Web UI	Formatted answer + sources	Streamlit, web apps
JSON	APIs	Answer + sources + metadata + confidence	REST APIs, microservices
HTML	Email/Reports	Rich formatted answer	Email responses, reports

Hallucination Detection in Post-Processing

Method	How it Works	Accuracy	Speed
Keyword overlap	Check if answer words exist in context	⭐⭐ Low	✅ Fast
NLI (Natural Language Inference)	Model checks if context entails answer	⭐⭐⭐⭐ High	⚠️ Medium
LLM-as-judge	Another LLM verifies the answer	⭐⭐⭐⭐⭐ Best	❌ Slow, costly
Groundedness score	% of answer words found in context	⭐⭐⭐ Medium	✅ Fast

Confidence Score Interpretation

Confidence	Meaning	Action
High (>0.8)	Answer well-supported by context	Show answer confidently
Medium (0.5-0.8)	Partially supported	Show answer with caveat
Low (<0.5)	Poorly supported	Show "I'm not sure" or ask for clarification

Common LLM Filler Phrases to Remove

- "As an AI language model..."
- "Based on the provided context..."
- "According to the given information..."
- "I think..."
- "It's important to note that..."
- "In summary..."

Step 15: Evaluation & Monitoring

What is it?

Measure how well your RAG pipeline performs across retrieval quality, answer quality, and system health. This is essential for production systems.

Evaluation Categories

A. Retrieval Evaluation Metrics

Metric	What it Measures	Formula	Range	Good Score
Precision@K	% of retrieved chunks that are relevant	relevant_retrieved / K	0-1	> 0.7
Recall@K	% of all relevant chunks that were retrieved	relevant_retrieved / total_relevant	0-1	> 0.8
MRR	How high the first relevant result ranks	1/rank_of_first_relevant	0-1	> 0.7
MAP	Average precision across all queries	mean(AP per query)	0-1	> 0.6
NDCG	Ranking quality (considers position)	DCG/IDCG	0-1	> 0.7
Hit Rate	% of queries where at least 1 relevant chunk found	queries_with_hit / total_queries	0-1	> 0.9

B. Generation Evaluation Metrics

Metric	What it Measures	How to Compute	Range	Good Score
Relevance	Is the answer relevant to the question?	Keyword overlap or LLM judge	0-1	> 0.8
Groundedness	Is the answer based on the context?	% answer words in context	0-1	> 0.7
Faithfulness	Does the answer contradict the context?	NLI model or LLM judge	0-1	> 0.85
BLEU	N-gram overlap with reference answer	BLEU score	0-1	> 0.3
ROUGE-L	Longest common subsequence with reference	ROUGE score	0-1	> 0.4
BERTScore	Semantic similarity with reference	BERT embeddings	0-1	> 0.8

C. End-to-End Metrics

Metric	What it Measures	Good Score
Answer Correctness	Is the final answer correct?	> 0.85
Latency	Total time from query to answer	< 3 seconds

Metric	What it Measures	Good Score
Token Usage	Tokens consumed per query	< 2000
Cost per Query	$ spent per query	< $0.01
User Satisfaction	Thumbs up/down ratio	> 80% positive

Evaluation Frameworks

Framework	What it Does	Metrics Provided	Cost
RAGAS	RAG-specific evaluation	Faithfulness, relevance, context precision	Free
DeepEval	LLM evaluation framework	Hallucination, bias, toxicity	Free
TruLens	Feedback functions for LLM apps	Groundedness, relevance, sentiment	Free
LangSmith	LangChain's evaluation platform	Traces, feedback, datasets	Paid
Custom (our approach)	Keyword-based metrics	Relevance, groundedness, coverage	Free

Our Evaluation Metrics Explained

Relevance Score

```
relevance = |query_words ∩ chunk_words| / |query_words|
```

- Measures: How many query keywords appear in retrieved chunks
- **Good**: > 0.7 | **Bad**: < 0.4

Groundedness Score

```
groundedness = |answer_words ∩ context_words| / |answer_words|
```

- Measures: How much of the answer comes from the context
- **Good**: > 0.7 | **Bad**: < 0.5 (likely hallucinating)

Chunk Coverage

```
coverage = chunks_that_contributed / total_chunks_retrieved
```

- Measures: How many retrieved chunks actually helped the answer
- **Good**: > 0.5 | **Bad**: < 0.2 (retrieving too many irrelevant chunks)

Monitoring in Production

What to Monitor	Why	Alert Threshold
Latency (p50, p95, p99)	User experience	p95 > 5s
Error rate	System health	> 1%
Relevance score trend	Retrieval quality degradation	Drops below 0.6
Groundedness trend	Hallucination increase	Drops below 0.6
Cache hit rate	Optimization effectiveness	< 20% (cache not helping)

What to Monitor	Why	Alert Threshold
Token usage per query	Cost control	> 5000 tokens avg
Unique queries per day	Usage patterns	Sudden spikes/drops

Offline vs Online Evaluation

Aspect	Offline	Online
When	Before deployment	After deployment
Data	Test dataset with ground truth	Real user queries
Metrics	Precision, recall, BLEU, ROUGE	Latency, user satisfaction, error rate
Purpose	Validate pipeline quality	Monitor production health
Frequency	Before each release	Continuous

Step 16: Caching & Optimization

What is it?

Reduce latency, cost, and redundant computation by caching results and optimizing the retrieval-generation pipeline.

Types of Caching

Cache Type	Where	Speed	Persistence	Best For
In-Memory	RAM	⚡ <1ms	❌ Lost on restart	Hot queries, single session
Disk Cache	Local files	⚡ 1-5ms	✅ Survives restart	Repeated queries across sessions
Redis	External server	⚡ 1-5ms	✅ Persistent	Production, distributed systems
Semantic Cache	Vector DB	⚠️ 10-50ms	✅ Persistent	Similar (not exact) queries

Cache Strategy Comparison

Strategy	How it Works	Hit Rate	Complexity
Exact Match	Hash query → lookup	⭐⭐ Low (exact only)	✅ Simple
Semantic Cache	Embed query → find similar cached query	⭐⭐⭐⭐ High	⚠️ Medium
Two-Tier (Memory + Disk)	Check RAM first, then disk	⭐⭐⭐ Medium	⚠️ Medium
TTL-based	Expire after N seconds	Depends	✅ Simple

TTL (Time-To-Live) Guidelines

Data Type	Recommended TTL	Why
Static docs (policies, manuals)	24-72 hours	Content rarely changes
News/articles	1-6 hours	Content updates frequently

Data Type	Recommended TTL	Why
Real-time data (stock, weather)	1-5 minutes	Stale data is harmful
User-specific queries	30-60 minutes	Session-based relevance

Optimization Techniques

Technique	What it Does	Impact	Complexity
Deduplication	Remove near-duplicate chunks before LLM	Reduces tokens, less noise	✅ Easy
Re-ranking	Reorder chunks by relevance	Better answer quality	✅ Easy
Context Compression	Trim chunks to fit token budget	Reduces cost	✅ Easy
Embedding Caching	Cache embeddings for repeated documents	Faster ingestion	⚠️ Medium
Query Caching	Cache query embeddings	Faster retrieval	✅ Easy
Result Caching	Cache final answers	Fastest response	✅ Easy
Async Processing	Parallel embedding + retrieval	Lower latency	⚠️ Medium
Batch Queries	Process multiple queries together	Higher throughput	⚠️ Medium

Optimization Impact

Optimization	Latency Reduction	Cost Reduction	Quality Impact
Result caching	90-99% (cache hit)	90-99%	None (same answer)
Deduplication	10-20%	10-30%	✅ Improves (less noise)
Re-ranking	0% (adds time)	0%	✅ ✅ Improves significantly
Context compression	5-15%	20-40%	⚠️ May slightly reduce
Semantic caching	70-90% (similar queries)	70-90%	⚠️ Slight variation

Cache Invalidation Strategies

Strategy	When to Invalidate	Pros	Cons
TTL expiry	After fixed time	Simple, predictable	May serve stale data
On data update	When source documents change	Always fresh	Complex to implement
Manual	Admin triggers clear	Full control	Requires human action
LRU (Least Recently Used)	When cache is full, remove oldest	Memory efficient	May evict useful entries

---

Step 17: Deploy the RAG Application (API / UI)

What is it?

Make your RAG pipeline accessible to users through an API endpoint, web interface, or both.

Deployment Options

Option	Framework	Best For	Complexity	Scalability
REST API	FastAPI	Backend services, integrations	⚠️ Medium	✅ High
REST API	Flask	Simple APIs	✅ Easy	⚠️ Medium
Web UI	Streamlit	Demos, internal tools	✅ Easy	❌ Low
Web UI	Gradio	ML demos, quick prototypes	✅ Easy	❌ Low
Full Web App	React + FastAPI	Production user-facing apps	❌ Complex	✅ High
Serverless	AWS Lambda + API Gateway	Event-driven, pay-per-use	⚠️ Medium	✅ ✅ Auto-scale
Container	Docker + ECS/EKS	Enterprise, microservices	❌ Complex	✅ ✅ High

Framework Comparison

Feature	FastAPI	Flask	Streamlit	Gradio
Speed	✅ Async, fast	⚠️ Sync	❌ Not for APIs	❌ Not for APIs
Auto docs	✅ Swagger built-in	❌ Manual	N/A	N/A
UI	❌ No (API only)	❌ No (API only)	✅ Built-in	✅ Built-in
Production ready	✅ Yes	✅ Yes	⚠️ Limited	⚠️ Limited
Learning curve	⚠️ Medium	✅ Easy	✅ Easy	✅ Easy

API Design Best Practices

Endpoint	Method	Purpose
`POST /ingest`	POST	Add new documents to knowledge base
`POST /query`	POST	Ask a question
`GET /stats`	GET	Get pipeline metrics
`POST /clear-cache`	POST	Clear cached results
`GET /health`	GET	Health check
`DELETE /documents/{id}`	DELETE	Remove a document

AWS Deployment Architecture

```
User → CloudFront (CDN) → API Gateway → Lambda / ECS
                                        ↓
                                  RAG Pipeline:
                                  - Bedrock (LLM + Embeddings)
                                  - OpenSearch (Vector DB)
                                  - S3 (Document Storage)
                                  - ElastiCache Redis (Caching)
                                  - CloudWatch (Monitoring)
```

Deployment Checklist

Category	Item	Priority
Security	API key / authentication	🔴 Critical
Security	Rate limiting	🔴 Critical
Security	Input validation / sanitization	🔴 Critical
Security	HTTPS only	🔴 Critical
Performance	Caching layer	🟡 High
Performance	Async processing	🟡 High
Performance	Connection pooling	🟡 High
Reliability	Health checks	🟡 High
Reliability	Error handling & retries	🟡 High
Reliability	Logging & monitoring	🟡 High
Scalability	Horizontal scaling (multiple instances)	🟢 Medium
Scalability	Load balancing	🟢 Medium
Operations	CI/CD pipeline	🟢 Medium
Operations	Environment variables for secrets	🔴 Critical

Scaling Strategies

Scale	Users	Architecture	Cost
Prototype	1-10	Single machine, Streamlit + FAISS	Free
Small	10-100	FastAPI + ChromaDB + OpenAI	$50-200/mo
Medium	100-10K	Docker + Pinecone + OpenAI + Redis	$500-2000/mo
Large	10K-100K+	ECS/EKS + OpenSearch + Bedrock + ElastiCache	$2000-10K+/mo

---

Complete Pipeline Summary

```
Step 1:  Define Use Case          → What problem are we solving?
Step 2:  Collect Data             → PDF, web, API, database
Step 3:  Clean & Preprocess       → Remove noise, normalize
Step 4:  Parse Documents          → Extract text, tables, metadata
Step 5:  Chunk Text               → Split into retrievable pieces
Step 6:  Generate Embeddings      → Convert chunks to vectors
Step 7:  Store in Vector DB       → FAISS, Chroma, Pinecone
Step 8:  Query Embedding          → Convert user question to vector
Step 9:  Similarity Search        → Find closest chunks
Step 10: Top-K Retrieval          → Select best K chunks
Step 11: Build Prompt             → Combine context + question
Step 12: Send to LLM             → OpenAI, Bedrock, Ollama
Step 13: Generate Answer          → LLM produces response
Step 14: Post-Process             → Clean, format, add sources
Step 15: Evaluate & Monitor       → Measure quality, track metrics
```

```
Step 16: Cache & Optimize         → Speed up, reduce costs
Step 17: Deploy                   → API + UI for users
```

Quick Reference: Recommended Stack by Scale

Component	Prototype	Production (Small)	Production (Large)
Embedding	TF-IDF	all-MiniLM-L6-v2	OpenAI text-embedding-3-small
Vector DB	FAISS	ChromaDB	Pinecone / OpenSearch
LLM	Ollama (free)	GPT-4o-mini	GPT-4o / Claude 3.5
Chunking	Word-based	Recursive	Semantic
Caching	In-memory	Disk	Redis
API	None	FastAPI	FastAPI + API Gateway
UI	Streamlit	Streamlit	React
Monitoring	Print logs	File logs	CloudWatch / Datadog

Notes generated as part of RAG Pipeline learning project